

Ideation Phase

Define the Problem Statements

Date	3 October 2026
Team ID	SWTID-2026-1299
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Marks

ServiceNow Incident Management Project - Milestone Documentation & Template

Create a problem statement to understand your customer's point of view. The Customer Problem Statement template helps you focus on what matters to create experiences people will love.

A well-articulated customer problem statement allows you and your team to find the ideal solution for the challenges your customers face. Throughout the process, you'll also be able to empathize with your customers, which helps you better understand how they perceive your product or service.

Customer Problem Statement Framework

I am	<i>Describe customer with 3-4 key characteristics - who are they?</i>	Describe the customer and their attributes here
I'm trying to	<i>List their outcome or "job" the care about - what are they trying to achieve?</i>	List the thing they are trying to achieve here
but	<i>Describe what problems or barriers stand in the way – what bothers them most?</i>	Describe the problems or barriers that get in the way here
because	<i>Enter the "root cause" of why the problem or barrier exists – what needs to be solved?</i>	Describe the reason the problems or barriers exist
which makes me feel	<i>Describe the emotions from the customer's point of view – how does it impact them emotionally?</i>	Describe the emotions the result from experiencing the problems or barriers

Reference: <https://miro.com/templates/customer-problem-statement/>

Project Milestones & Problem Statement Matrix

Problem Statement (PS)	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1 (Milestone 1)	ServiceNow Admin / Developer	Create a UI Policy on the Incident table to dynamically control the behavior of form fields based on specific conditions. Configure the UI Policy to make selected fields mandatory, read-only, or visible when the defined condition is met.	Users enter inconsistent data or bypass required field updates.	Dynamic validations, field controls, and client-side automation are missing.	Concerned about data integrity and operational efficiency.
PS-2 (Milestone 2)	ServiceNow Admin / Developer	Create a UI Policy Action for the Urgency field on the Incident form to control its behavior based on the conditions defined in the UI Policy. Configure the action to make the Urgency field mandatory, visible, or read-only as required.	Users enter inconsistent data or bypass required field updates.	Dynamic validations, field controls, and client-side automation are missing.	Concerned about data integrity and operational efficiency.
PS-3 (Milestone 3)	ServiceNow Admin / Developer	Create an onChange Client Script on the Incident form to dynamically perform an action whenever a specified field value is changed. The script should use the g_form API to update related fields or display appropriate messages based on the selected value.	Users enter inconsistent data or bypass required field updates.	Dynamic validations, field controls, and client-side automation are missing.	Concerned about data integrity and operational efficiency.
PS-4 (Milestone 4)	ServiceNow Admin / Developer	Create an onSubmit Client Script on the Incident form to validate the information entered by the user before the record is saved or submitted. Use the g_form API to check whether the	Users enter inconsistent data or bypass required field updates.	Dynamic validations, field controls, and client-side automation are missing.	Concerned about data integrity and operational efficiency.

		required conditions are satisfied and prevent submission when invalid or incomplete data is found. Display an appropriate message to guide the user in correcting the information before saving the Incident.			
PS-5 (Milestone 5)	ServiceNow Admin / Developer	Create an onCellEdit Client Script on the Incident table to control and validate field values when users edit Incident records directly from a list view. The script should check the updated value and allow or prevent the change based on the defined conditions.	Users enter inconsistent data or bypass required field updates.	Dynamic validations, field controls, and client-side automation are missing.	Concerned about data integrity and operational efficiency.
PS-6 (Milestone 6)	ServiceNow Admin / Developer	Test all the configured UI Policies, UI Policy Actions, and Client Scripts on the Incident form and list view to ensure they work as expected. Verify that fields become mandatory, visible, read-only, or updated based on the defined conditions. Test different scenarios, including valid and invalid inputs, form submission, field changes, and inline list edits, and confirm that the expected validation messages and behaviors are triggered correctly.	Users enter inconsistent data or bypass required field updates.	Dynamic validations, field controls, and client-side automation are missing.	Concerned about data integrity and operational efficiency.

Detailed Project Milestones (Verbatim)

Milestone 1: Create a UI Policy on the Incident table to dynamically control the behavior of form fields based on specific conditions. Configure the UI Policy to make selected fields mandatory, read-only, or visible when the defined condition is met. This helps ensure that users provide the required information and follow consistent data-entry practices while creating or updating Incident records.

Milestone 2: Create a UI Policy Action for the Urgency field on the Incident form to control its behavior based on the conditions defined in the UI Policy. Configure the action to make the Urgency field mandatory, visible, or read-only as required. This ensures that the appropriate urgency information is captured consistently and helps users prioritize incidents accurately.

Milestone 3: Create an onChange Client Script on the Incident form to dynamically perform an action whenever a specified field value is changed. The script should use the `g_form` API to update related fields or display appropriate messages based on the selected value. This helps automate form behavior, improve data accuracy, and provide users with a more interactive Incident form experience.

Milestone 4: Create an onSubmit Client Script on the Incident form to validate the information entered by the user before the record is saved or submitted. Use the `g_form` API to check whether the required conditions are satisfied and prevent submission when invalid or incomplete data is found. Display an appropriate message to guide the user in correcting the information before saving the Incident.

Milestone 5: Create an onCellEdit Client Script on the Incident table to control and validate field values when users edit Incident records directly from a list view. The script should check the updated value and allow or prevent the change based on the defined conditions. This helps maintain data accuracy and ensures that users follow the required validation rules even when making inline edits from the Incident list.

Milestone 6: Test all the configured UI Policies, UI Policy Actions, and Client Scripts on the Incident form and list view to ensure they work as expected. Verify that fields become mandatory, visible, read-only, or updated based on the defined conditions. Test different scenarios, including valid and invalid inputs, form submission, field changes, and inline list edits, and confirm that the expected validation messages and behaviors are triggered correctly.